

Clevis Drawing Tool - GUI User Manual

Version 1.0

Date: December 2025

Table of Contents

<i>Clevis Drawing Tool - GUI User Manual.....</i>	<i>1</i>
<i>1. Introduction.....</i>	<i>2</i>
<i>2. Getting Started</i>	<i>3</i>
Launching the Application	3
First Steps	3
<i>3. Main Interface Overview</i>	<i>4</i>
3.1 Toolbar.....	4
3.2 Drawing Canvas.....	5
3.3 Side Panel	6
3.4 Command Console	7
<i>4. Complete Command Reference</i>	<i>7</i>
4.1 Shape Creation Commands	7
4.2 Grouping Commands.....	12
4.3 Editing Commands	14
4.4 Information Commands.....	16
4.5 History Commands	20
4.6 System Commands	22
<i>5. Advanced Features & Important Notes</i>	<i>22</i>
5.1 Name Conflict Resolution During Ungrouping.....	22
5.2 Undo/Redo System	24
5.3 Bounding Box Visualization	24
5.4 Group Management	25
5.5 Synchronization Features	25
<i>6. Troubleshooting.....</i>	<i>26</i>

Synchronization Issues	26
Performance Optimization	27
Common Error Messages	27
Performance Optimization	28
Data Persistence	28
Quick Reference Card	28
Essential Commands	29
Information Commands	29
System Commands	29
Critical Reminders	29

1. Introduction

Welcome to Clevis Drawing Tool! This graphical interface allows you to create and manipulate geometric shapes through both visual controls and command-line inputs.

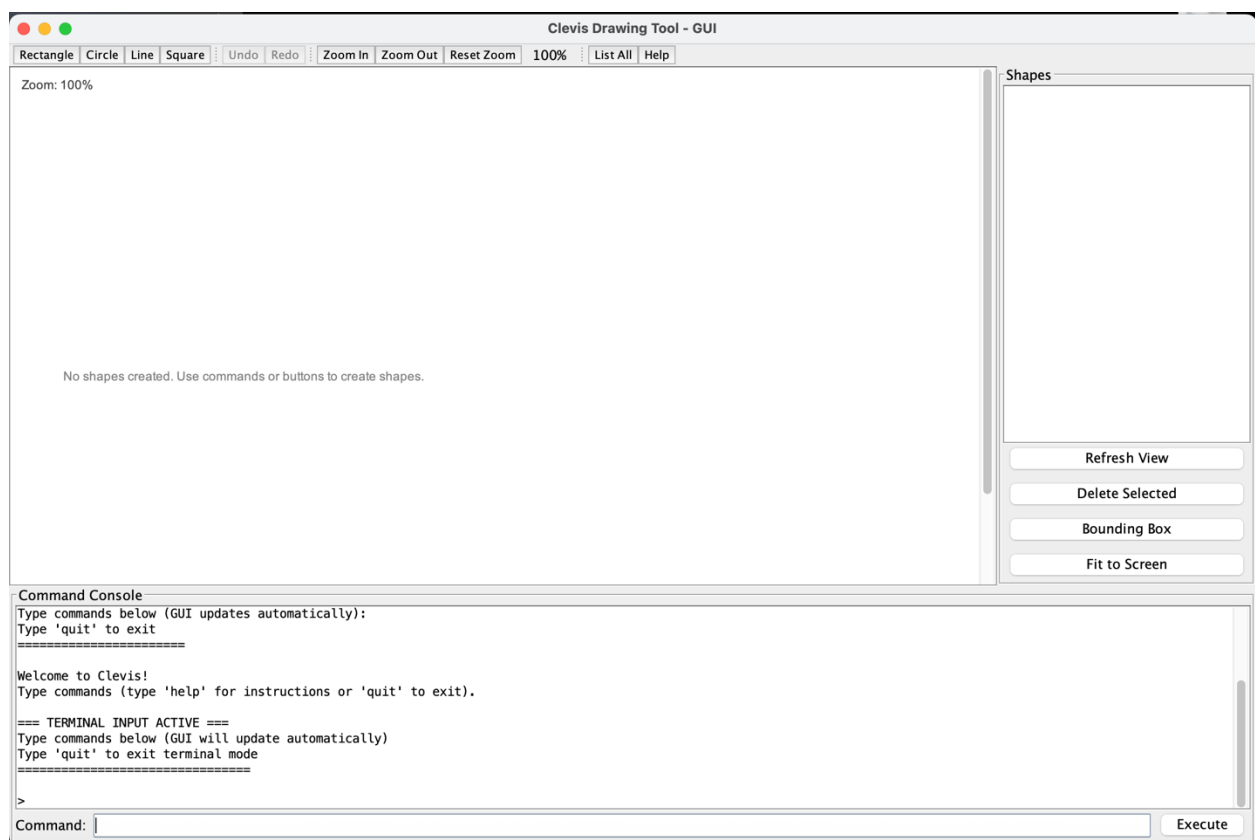


Figure 1: Main Clevis GUI Interface showing all major components

2. Getting Started

Launching the Application

When you start Clevis, you can interact through:

1. **GUI Main Window** with visual controls and command console
2. **Terminal Window** for command-line input (optional)

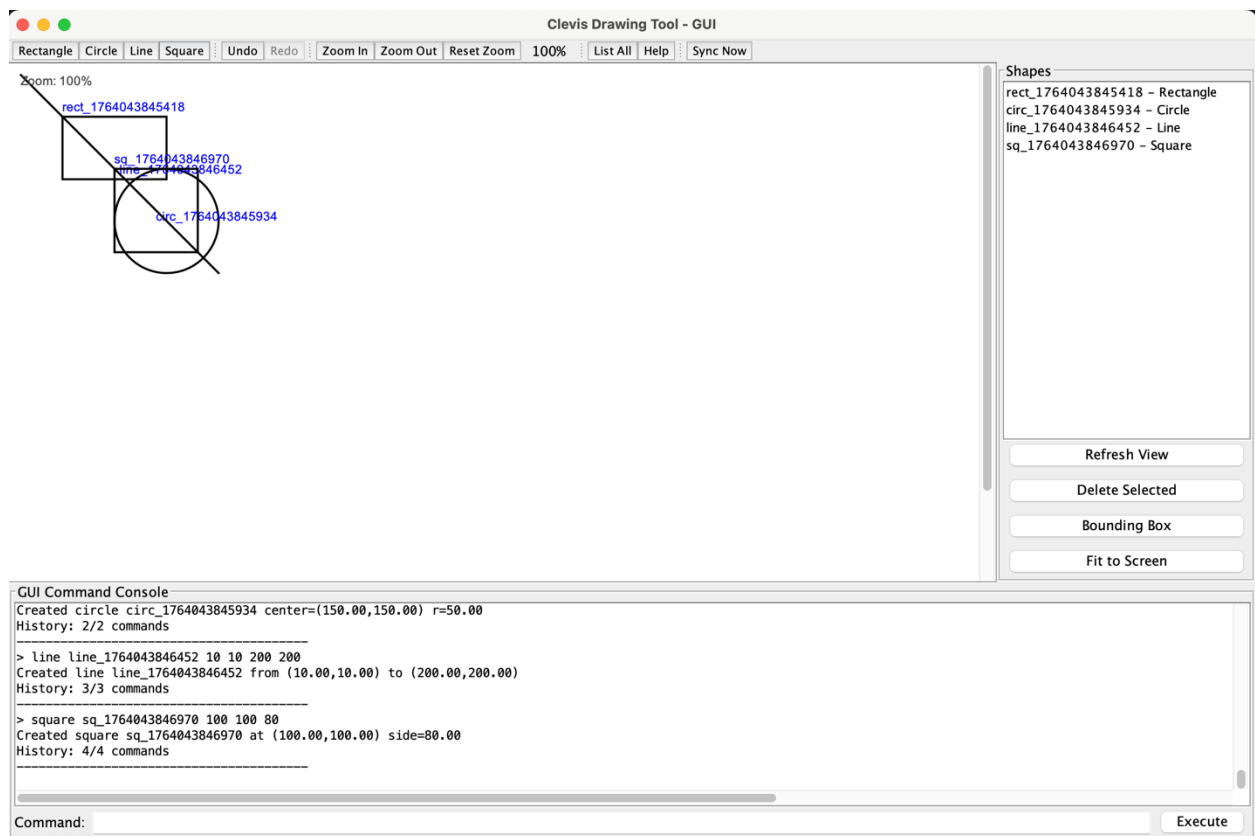


Figure 2: Clevis GUI Interface with command console and drawing canvas

First Steps

1. All commands work in both GUI command console and terminal
2. GUI updates automatically when commands execute
3. Use toolbar for quick actions or commands for precise control

3. Main Interface Overview

3.1 Toolbar

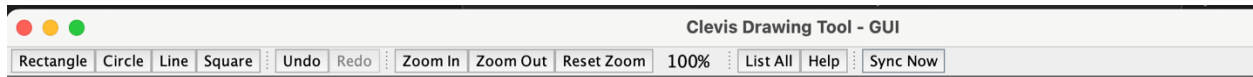


Figure 3: Toolbar Layout with Functional Groups

Shape Creation Buttons:

- **Rectangle:** Creates rectangle with auto-generated name
- **Circle:** Creates circle with auto-generated name
- **Line:** Creates line with auto-generated name
- **Square:** Creates square with auto-generated name

Edit Operations:

- **Undo:** Reverts last mutable command
- **Redo:** Re-applies last undone command

View Controls:

- **Zoom In:** Increases view magnification
- **Zoom Out:** Decreases view magnification
- **Reset Zoom:** Returns to default 100% view
- **Zoom Indicator:** Shows current zoom percentage

Utility Buttons:

- **List All:** Displays all shapes in console
- **Help:** Shows comprehensive command guide
- **Sync Now:** Forces immediate synchronization between terminal and GUI

3.2 Drawing Canvas

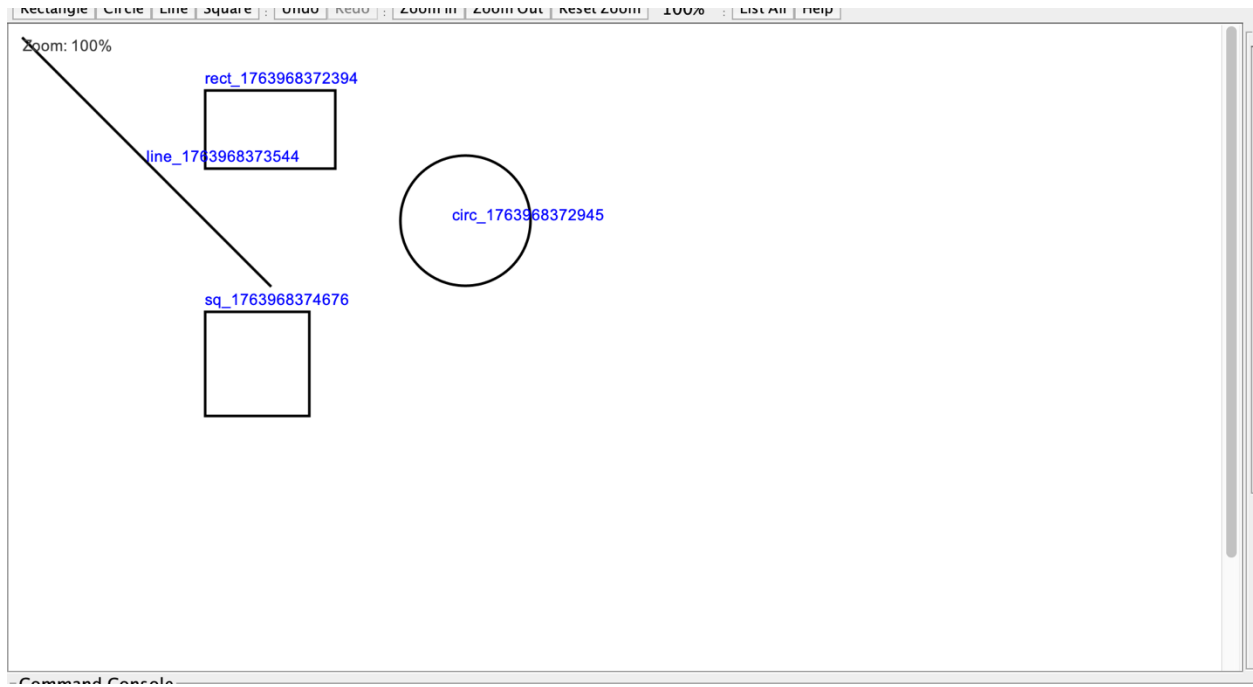


Figure 4: Drawing Canvas with Multiple Shape Types

Features:

- Real-time shape rendering
- Automatic updates on changes
- Bounding box visualization
- Scrollable viewing area

3.3 Side Panel

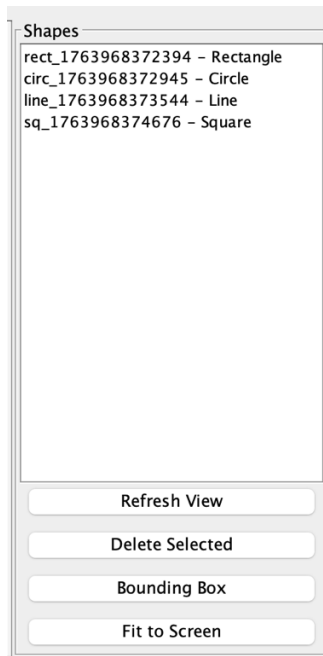


Figure 5: Side Panel with Shape List and Controls

Shapes List:

- Shows all shapes with names and types
- Click to select shapes for operations
- Groups display as single entries

Control Buttons:

- **Refresh View:** Clears highlights and redraws
- **Delete Selected:** Removes selected shape
- **Bounding Box:** Shows bounding box for selected shape
- **Fit to Screen:** Auto-zooms to show all content

3.4 Command Console

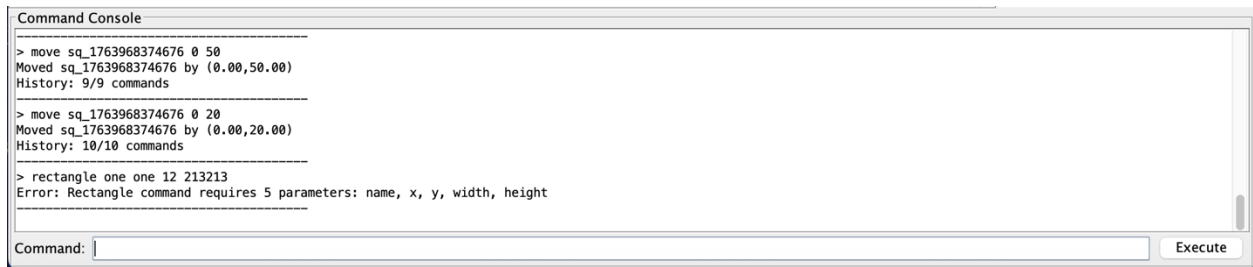


Figure 6: Command Console with Input and Output Display

Components:

- **Command History:** Shows commands and results
- **Command Input:** Type commands here
- **Execute Button:** Runs entered command

4. Complete Command Reference

4.1 Shape Creation Commands

Rectangle

clevis

rectangle <name> <x> <y> <width> <height>

Parameters:

- name: Unique identifier for the shape
- x, y: Top-left corner coordinates
- width, height: Dimensions of the rectangle

Example:

clevis

rectangle myRect 50 50 100 60

Output:

text

Created a Rectangle named myRect at (50.00,50.00) w=100.00 h=60.00

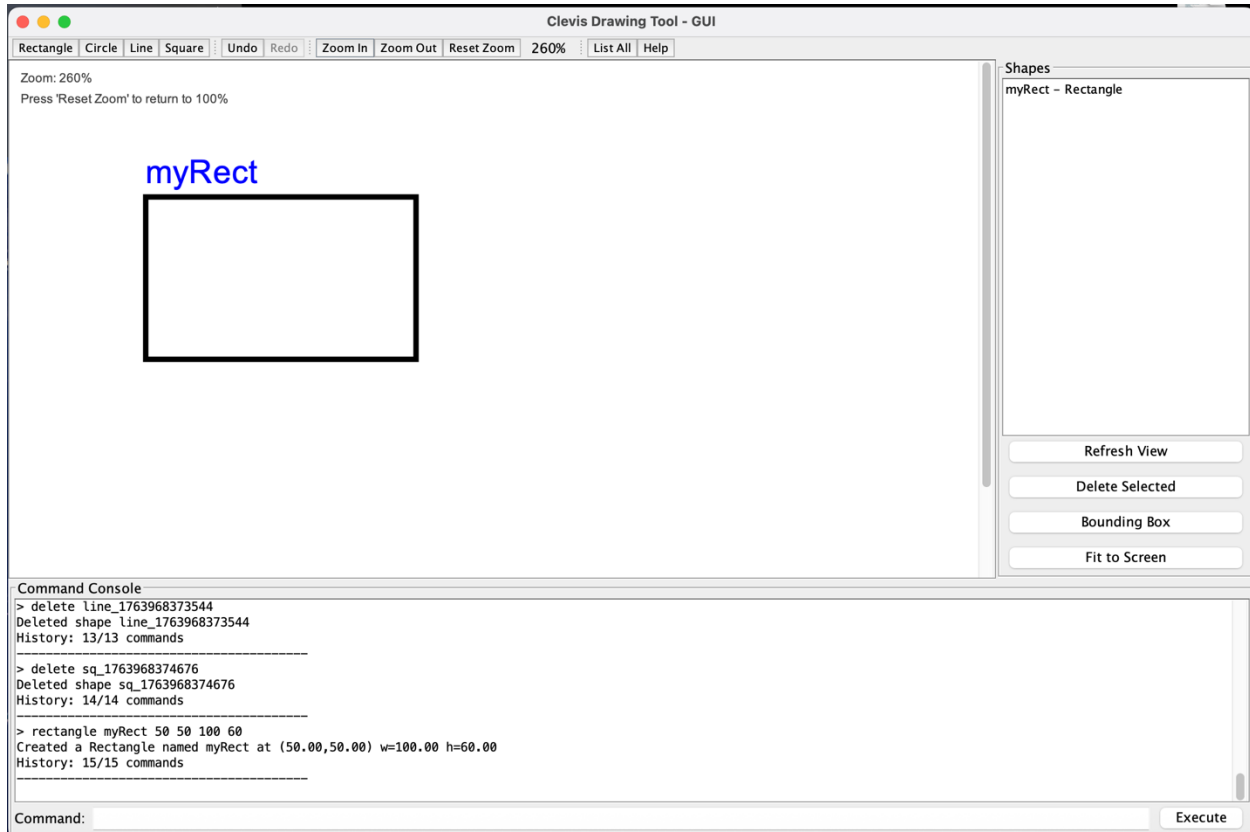


Figure 7: Rectangle Creation Example

Important Notes:

- Names must be unique across all shapes
- Coordinates and dimensions can be decimal values
- Negative dimensions are not allowed

Circle

clevis

circle <name> <x> <y> <radius>

Parameters:

- name: Unique shape identifier
- x, y: Center point coordinates
- radius: Circle radius

Example:

clevis

circle myCircle 150 150 45.5

Output:

text

Created circle myCircle center=(150.00,150.00) r=45.50

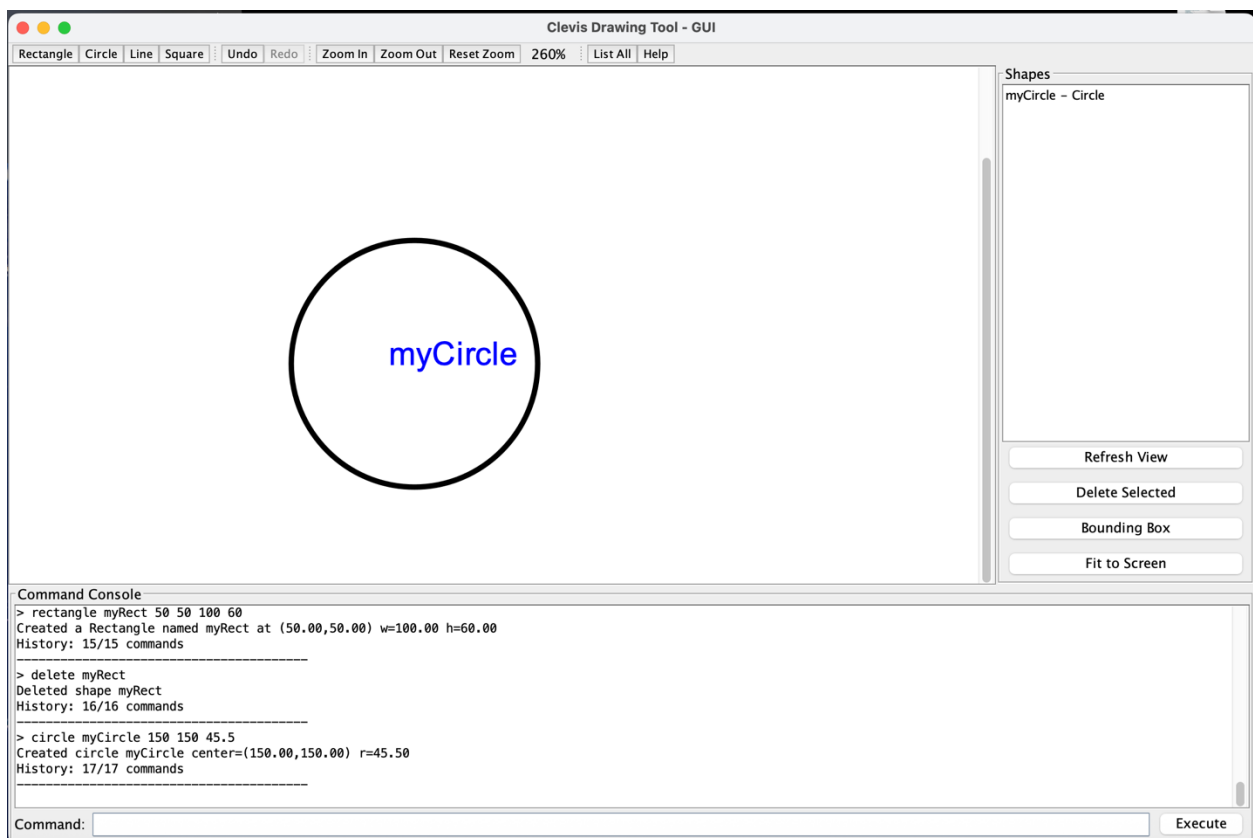


Figure 8: Circle Creation Example

Line

clevis

line <name> <x1> <y1> <x2> <y2>

Parameters:

- name: Unique shape identifier
- x1, y1: Starting point coordinates
- x2, y2: Ending point coordinates

Example:

clevis

line connector 10 10 200 150

Output:

text

Created line connector from (10.00,10.00) to (200.00,150.00)

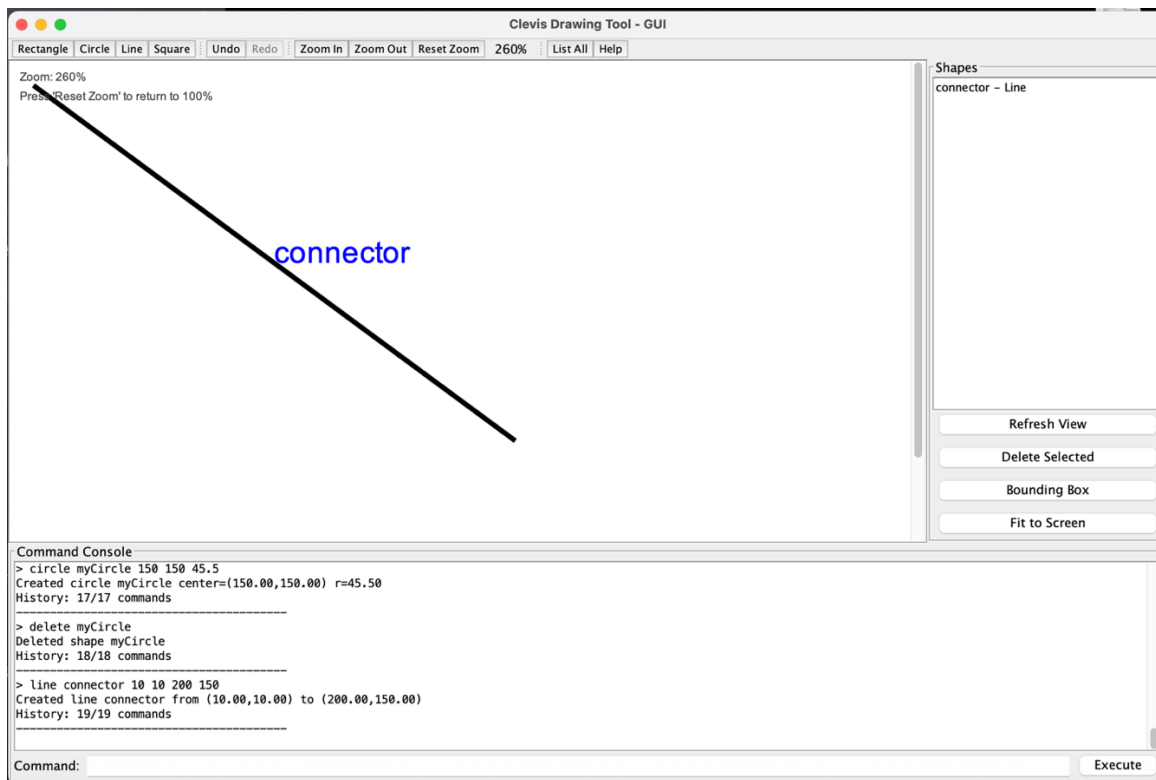


Figure 9: Line Creation Example

Square

clevis

square <name> <x> <y> <length>

Parameters:

- name: Unique shape identifier
- x, y: Top-left corner coordinates
- length: Side length

Example:

clevis

square mySquare 100 100 75

Output:

text

Created square mySquare at (100.00,100.00) side=75.00

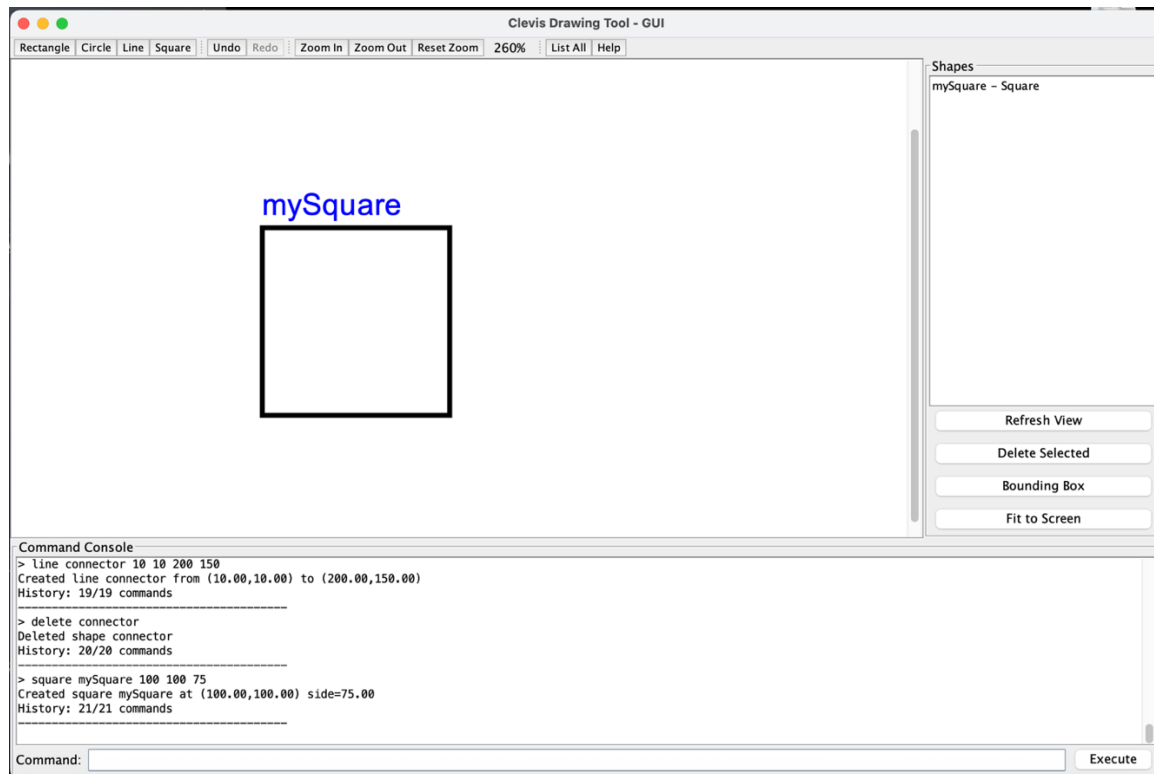


Figure 10: Square Creation Example

4.2 Grouping Commands

Group

clevis

group <group_name> <shape1> <shape2> ...

Parameters:

- group_name: Name for the new group
- shape1, shape2, ...: Names of shapes to group

Example:

clevis

wall1 wall2 door roof roof_left roof_right

Output:

text

Created group house containing: wall1 wall2 door roof roof_left roof_right

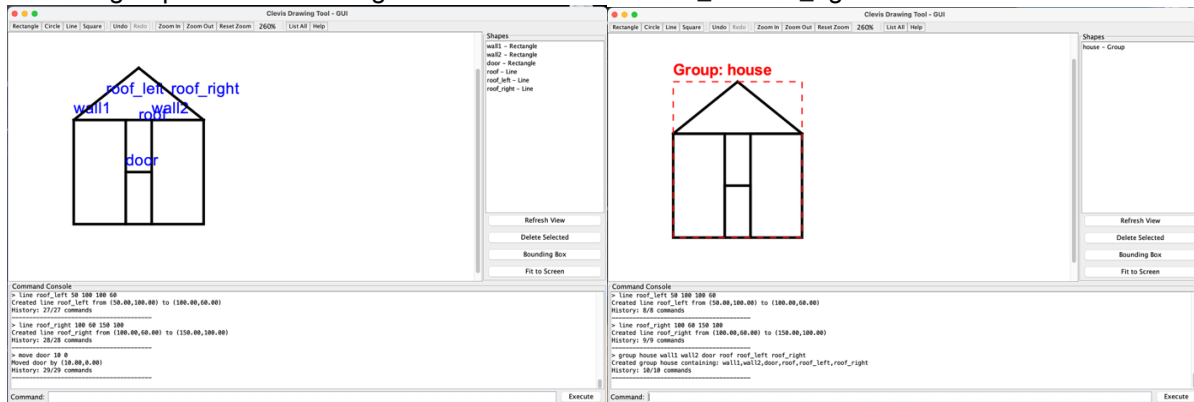


Figure 11: Grouping Multiple Shapes

Important Notes:

- Group name must not conflict with existing shapes
- At least two shapes required for grouping
- Original shapes are removed and replaced by group

Ungroup

clevis

ungroup <group_name>

Parameters:

- group_name: Name of group to ungroup

Example:

clevis

ungroup house

Output:

text

Ungrouped house into: wall1 wall2 door roof roof_left roof_right

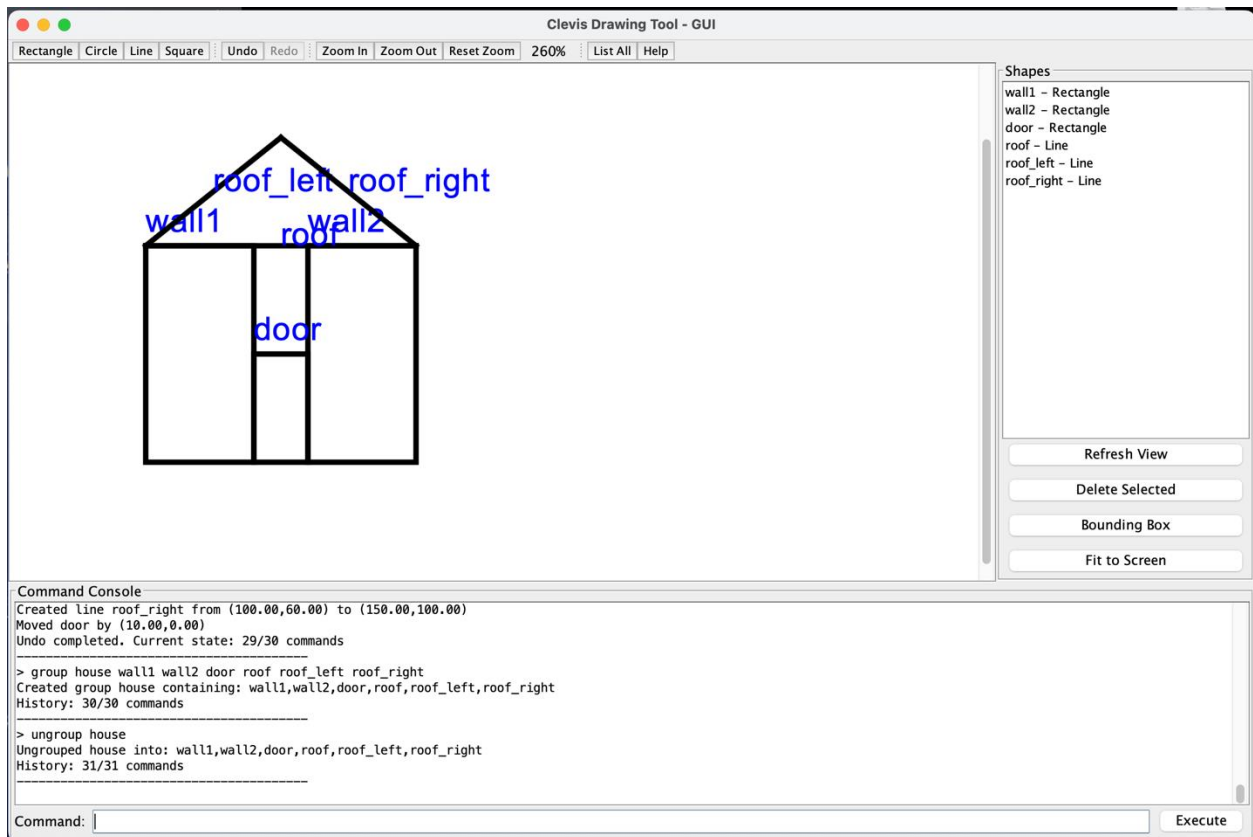


Figure 12: Ungroup Operation with Name Resolution

4.3 Editing Commands

Delete

clevis

delete <shape_name>

Parameters:

- shape_name: Name of shape to delete

Example:

clevis

delete wall1

Output:

text

Deleted shape wall1

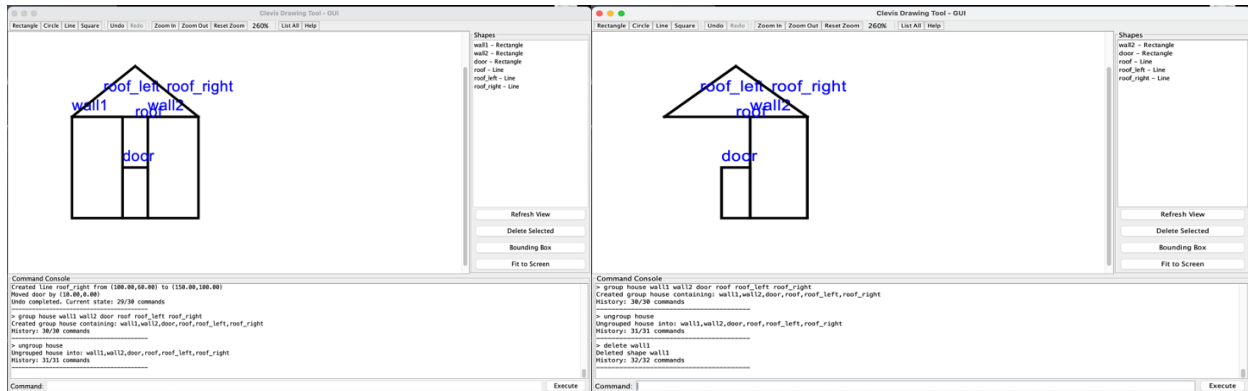


Figure 13: Shape Deletion Example

Important Notes:

- Deleting a group removes all member shapes
- Operation cannot be undone if followed by other commands

Move

clevvis

move <shape_name> <dx> <dy>

Parameters:

- shape_name: Name of shape to move
- dx: Horizontal displacement
- dy: Vertical displacement

Example:

clevvis

move door 15 -5

Output:

text

Moved door by (15.00,-5.00)

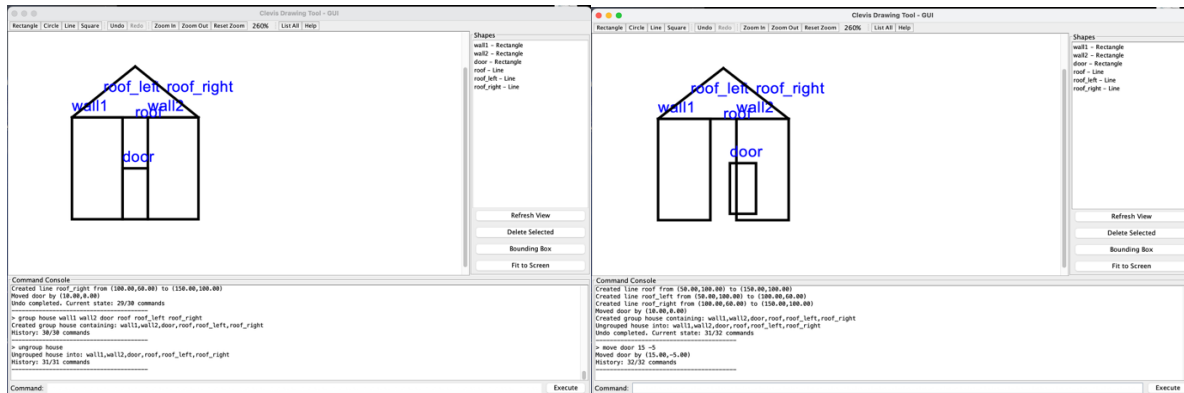


Figure 14: Shape Movement Visualization

4.4 Information Commands

List

clevis

list <shape_name>

Parameters:

- shape_name: Name of shape to inspect

Example:

clevis

list door

Output:

text

Shape door: Rectangle at (115.00,95.00) w=30.00 h=50.00


```
Command Console
Moved door by (10.00,0.00)
Created group house containing: wall1,wall2,door,roof,roof_left,roof_right
Ungrouped house into: wall1,wall2,door,roof,roof_left,roof_right
Undo completed. Current state: 31/32 commands

> move door 15 -5
Moved door by (15.00,-5.00)
History: 32/32 commands

> list door
Shape door: Rectangle[name:door,(x,y):(105.00,135.00),w:20.00,h:40.00]

Command:  Execute
```

Figure 15: List Command Output

List All

clevis

listAll

Example:

clevis

listAll

Output:

text

Listing all shapes (bottom to top):

-rectangle wall1 50 100 40 80

-rectangle wall2 110 100 40 80

-rectangle door 80 140 20 40

-line roof 50 100 150 100

-line roof_left 50 100 100 60

-line roof_right 100 60 150 100

```
Command Console
> list door
Shape door: Rectangle[name:door,(x,y):(105.00,135.00),w:20.00,h:40.00]

> listAll
Listing all shapes (bottom to top):
- Rectangle[name:wall1,(x,y):(50.00,100.00),w:40.00,h:80.00]
- Rectangle[name:wall2,(x,y):(110.00,100.00),w:40.00,h:80.00]
- Rectangle[name:door,(x,y):(105.00,135.00),w:20.00,h:40.00]
- Line[name:roof,(x1,y1):(50.00,100.00),(x2,y2):(150.00,100.00)]
- Line[name:roof_left,(x1,y1):(50.00,100.00),(x2,y2):(100.00,60.00)]
- Line[name:roof_right,(x1,y1):(100.00,60.00),(x2,y2):(150.00,100.00)]

Command:  Execute
```

Figure 16: ListAll Command Output

Bounding Box

clevis

boundingbox <shape_name>

Parameters:

- shape_name: Name of shape to analyze

Example:

clevis

boundingbox house

Output:

text

Bounding box of house: (x=50.00, y=60.00, width=100.00, height=120.00)

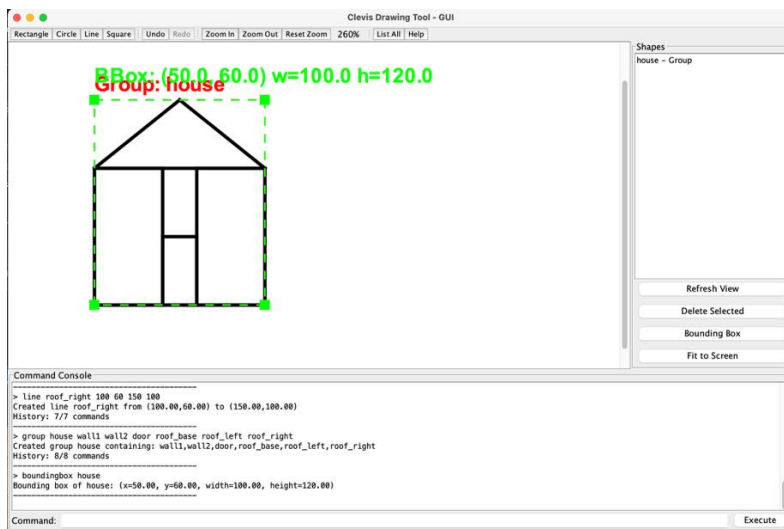


Figure 17: Bounding Box Visualization

Visual Effect: Green bounding box appears around the shape in GUI

Intersect

clevis

intersect <shape1> <shape2>

Parameters:

- shape1, shape2: Names of shapes to check

Example:

clevis

intersect wall1 wall2

Output:

text

Shapes wall1 and wall2 intersect: false

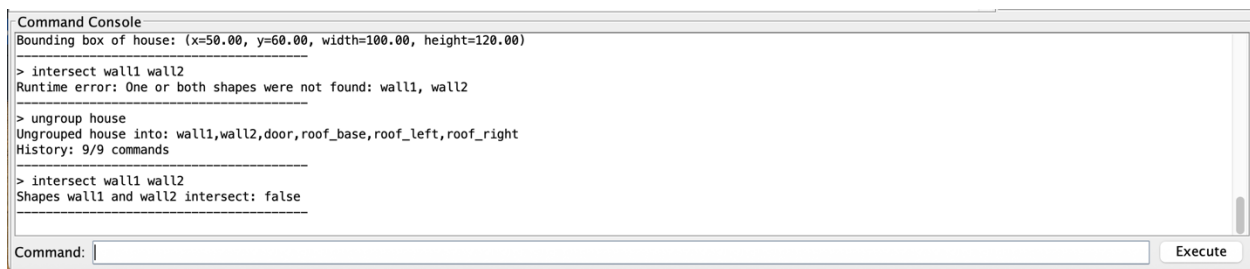


Figure 18: Intersection Check Example

Shape At Point

clevis

shapeAt <x> <y>

Parameters:

- x, y: Coordinates to check

Example:

clevis

shapeAt 75 75

Output:

text

The topmost shape covering point (50.00, 120.00) is: wall1

Command Console

> shapeAt 75 75

No shape covers the given point (75.0, 75.0).

> shapeAt 50 60

No shape covers the given point (50.0, 60.0).

> boundingbox wall1

Bounding box of wall1: (x=50.00, y=100.00, width=40.00, height=80.00)

> shapeAt 50 120

The topmost shape covering point (50.0, 120.0) is: wall1

Command:

Execute

Figure 19: ShapeAt Point Detection

4.5 History Commands

Undo

clevis
undo

Example:

clevis
undo

Output:

text
Undoing last command...
Undo completed. Current state: 6/9 commands

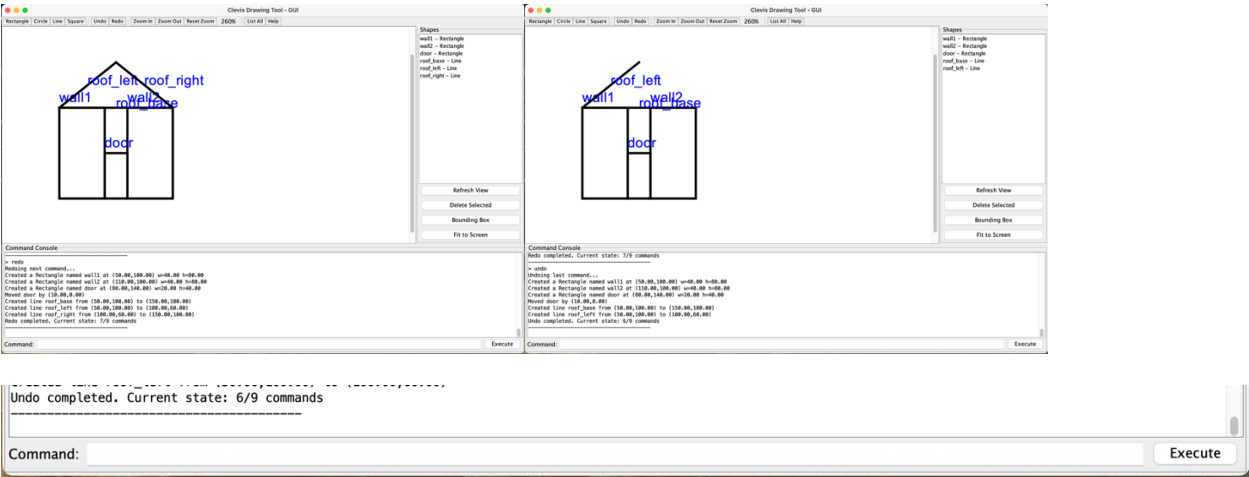


Figure 20: Undo Operation in Progress

Supported Operations:

- Shape creation (rectangle, circle, line, square)
- Grouping and ungrouping
- Deletion
- Movement

Not Supported:

- Information commands (list, boundingbox, etc.)
- Intersection checks

Redo

clevis
redo

Example:

clevis
redo

Output:

text
Redoing next command...
Redo completed. Current state: 7/9 commands

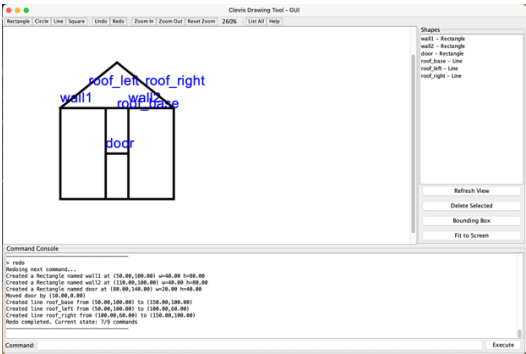




Figure 21: Redo Operation

4.6 System Commands

Help

clevis

help

Displays comprehensive command reference guide.

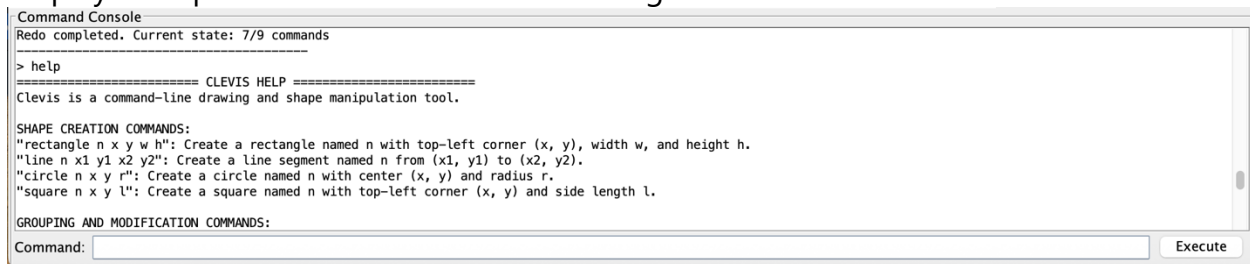


Figure 22: Help Command Output

Quit

clevis

quit

Exits the application and saves logs.

5. Advanced Features & Important Notes

5.1 Name Conflict Resolution During Ungrouping

Critical Behavior to Understand:

When ungrouping, if external shapes exist with the same names as group members, automatic renaming occurs:

Scenario:

1. Create shapes: shapeA, shapeB
2. Group them: group myGroup shapeA shapeB
3. Create new shape named shapeA
4. Ungroup: ungroup myGroup

What Happens:

text

Ungrouped myGroup into: shapeA_1,shapeB

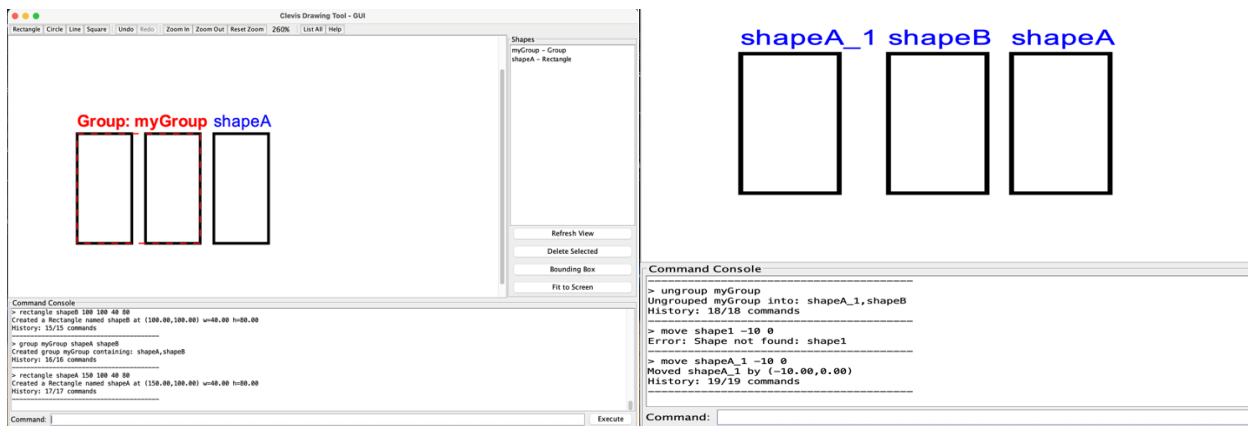


Figure 23: Automatic Renaming During Ungroup Operation

Why This Occurs:

- Clevis maintains unique naming for all shapes
- When ungrouping would cause name conflicts, it renames the initial shape
- This prevents accidental overwriting of existing shapes

Best Practices:

- Use descriptive, unique names from the start
- Check shape names with listAll before grouping operations

- Be aware that ungrouping may change existing shape names

5.2 Undo/Redo System

How It Works:

- Only mutable commands are stored in history
- History is maintained per session only
- Maximum history depth depends on available memory

Commands in History:

- ☒ rectangle, circle, line, square
- ☒ group, ungroup
- ☒ delete, move
- ☐ list, listAll, boundingbox, intersect, shapeAt

Status Display:

The undo/redo buttons show current state:

- **Undo enabled:** Highlighted, shows available undo
- **Undo disabled:** Grayed out, no actions to undo
- **Redo enabled:** Highlighted, shows available redo
- **Redo disabled:** Grayed out, no actions to redo

5.3 Bounding Box Visualization

Usage Tips:

- Bounding box appears as green outline around shape
- Use "Refresh View" to clear bounding box displays
- Bounding box coordinates are shown in console output

5.4 Group Management

Moving Groups:

When you move a group, all member shapes move together maintaining their relative positions.

Deleting Groups:

Deleting a group removes all member shapes permanently.

Nested Groups:

Nested groups are not allowed.

5.5 Synchronization Features

Sync Now Button

Purpose

The **Sync Now** button ensures immediate synchronization between the terminal interface and graphical interface when they become out of sync.

When to Use Sync Now

- After executing multiple commands in the terminal
- When shapes appear in different positions between terminal output and GUI display
- When the shape list doesn't match actual system state
- To clear all visual highlights and bounding boxes
- When undo/redo button states don't reflect current history

How It Works

Clicking **Sync Now** performs these actions simultaneously:

1. **Refreshes Shape List** - Updates side panel with current shapes
2. **Redraws Canvas** - Repaints all shapes with current positions
3. **Updates UI State** - Refreshes undo/redo button availability
4. **Clears Visual Effects** - Removes bounding boxes and highlights

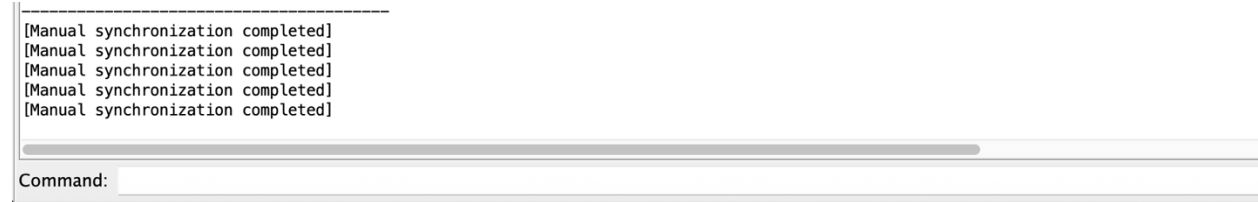


Figure 24: Sync Confirmation Message

Sync Confirmation

After clicking Sync Now, the command console displays:

text

[Manual synchronization completed]

Auto-Sync vs Manual Sync

- **Auto-Sync:** GUI automatically updates every second when changes are detected
- **Manual Sync:** Instant on-demand synchronization for critical updates

Best Practices

- Use Sync Now after batch terminal operations
- Click before precise visual measurements
- Use when switching between terminal and GUI intensive work
- Helpful for debugging shape states

6. Troubleshooting

Synchronization Issues

"GUI not showing terminal commands"

- **Cause:** Auto-sync delay or missed detection
- **Solution:** Click **Sync Now** button or wait 1-2 seconds for auto-sync

"Shapes in wrong positions between terminal and GUI"

- **Cause:** Visual state out of sync with data state
- **Solution:** Use **Sync Now** button for immediate correction

"Undo/redo buttons not updating after terminal commands"

- **Cause:** UI state not synchronized
- **Solution:** **Sync Now** button updates all interface elements

Performance Optimization

For Complex Drawings:

- Use **Sync Now** sparingly with 100+ shapes (performs full refresh)
- Rely on auto-sync for most operations
- Use groups to reduce the number of individual shapes
- "Refresh View" for canvas-specific issues, **Sync Now** for full interface sync

Common Error Messages

"Shape not found: shapeName"

- **Cause:** Typo in shape name or shape was deleted
- **Solution:** Use listAll to see available shapes

"Invalid number format"

- **Cause:** Non-numeric values in parameters
- **Solution:** Use only numbers for coordinates and dimensions

"Duplicate shape name: name"

- **Cause:** Shape name already exists

- **Solution:** Choose a different unique name

"Group must have at least one member"

- **Cause:** Group command with insufficient shapes
- **Solution:** Include at least two shape names

"Nothing to undo" / "Nothing to redo"

- **Cause:** No operations in history or history exhausted
- **Solution:** Execute mutable commands first

Performance Optimization

For Complex Drawings:

- Use groups to organize related shapes
- Delete unused shapes regularly
- Use "Refresh View" to clear visual overhead
- "Fit to Screen" helps navigate large drawings

Data Persistence

Automatic Logging:

- All commands are logged to text and HTML files
- Logs are saved in the logs directory
- Session history is not preserved between runs

Quick Reference Card

Essential Commands

rectangle name x y width height
circle name x y radius
line name x1 y1 x2 y2
square name x y length
group groupName shape1 shape2...
ungroup groupName
delete shapeName
move shapeName dx dy

Information Commands

list shapeName
listAll
boundingbox shapeName
intersect shape1 shape2
shapeAt x y

System Commands

undo
redo
help
quit

Critical Reminders

1. **Names must be unique** - no duplicates allowed
2. **Ungrouping may rename shapes** - check name conflicts
3. **Undo/redo has limits** - only certain commands are reversible
4. **Refresh View clears highlights** - use to reset visual state

Clevis Drawing Tool

Developed for COMP2021 Project

The Hong Kong Polytechnic University

© 2025 All Rights Reserved
